Java Day 9：static、静态变量与静态方法

学习目标：理解类级别共享成员，并用静态计数器完成性能测试请求统计

一、今天的学习目标

1. 理解普通成员变量与静态变量的区别。
2. 理解普通方法与静态方法的区别。
3. 使用 static 定义类级别共享变量。
4. 使用静态方法统计请求数量。
5. 理解 static 与性能测试统计程序之间的关系。

二、普通成员变量与静态变量

普通成员变量属于对象，每个对象各自保存一份。static 修饰的变量属于类，所有对象共享同一份。推荐使用类名访问静态变量，例如 HttpResult.totalCount。

```
public class HttpResult {
    private int responseTime;
    private int statusCode;

    public static int totalCount = 0;
}
```

每创建一个对象，可以在构造方法中执行 totalCount++。注意：这统计的是对象创建次数，不一定等于实际发送的请求数。

```
public HttpResult(int responseTime, int statusCode) {
    this.responseTime = responseTime;
    this.statusCode = statusCode;
    totalCount++;
}
```

三、静态方法与普通方法

普通方法通常依赖具体对象，例如 isSlow() 需要读取当前对象的 responseTime。静态方法属于类，可以通过类名调用，例如 HttpResult.showStatistics()。

```
public boolean isSlow() {
    return responseTime > 500;
}

public static void showTotalCount() {
    System.out.println(totalCount);
}
```

静态方法不能直接访问实例成员，因为调用静态方法时不一定有一个明确的对象。若要访问实例数据，可以把对象作为参数传入。

```
public static void showTime(HttpResult result) {
    System.out.println(result.responseTime);
}
```

四、性能测试实战：统计请求数量

本例统计对象总数、成功请求数和慢请求数。它用于理解 static 共享计数器；并发线程同时执行 int++ 时可能发生竞态，后续会学习线程安全计数。

HttpResult.java

```
package day9;

public class HttpResult {
    private int responseTime;
    private int statusCode;

    private static int totalCount = 0;
    private static int successCount = 0;
    private static int slowCount = 0;

    public HttpResult(int responseTime, int statusCode) {
        this.responseTime = responseTime;
        this.statusCode = statusCode;
        totalCount++;

        if (isSuccess()) {
            successCount++;
        }
        if (isSlow()) {
            slowCount++;
        }
    }

    public boolean isSuccess() {
        return statusCode >= 200 && statusCode < 300;
    }

    public boolean isSlow() {
        return responseTime > 500;
    }

    public static void showStatistics() {
        System.out.println("Total objects: " + totalCount);
        System.out.println("Successful: " + successCount);
        System.out.println("Slow: " + slowCount);
    }
}
```

Main.java

```
package day9;

public class Main {
    public static void main(String[] args) {
        new HttpResult(200, 200);
        new HttpResult(800, 500);
        new HttpResult(300, 200);
        new HttpResult(900, 200);
        new HttpResult(100, 404);

        HttpResult.showStatistics();
    }
}
```

正确输出：对象总数 5，成功请求数 3，慢请求数 2。状态码 500 和 404 都不属于 2xx。

五、关键知识点对比

对比	普通成员	static 成员
归属	对象	类
变量数量	每个对象各自一份	整个类共享一份
典型访问	对象名.成员	类名.静态成员
静态方法访问实例属性	不适用	不能直接访问

六、课后作业

作业 1：创建 Student 类，包含普通属性 name、age，以及静态属性 studentCount。每创建一个 Student 对象，studentCount 自动加 1。创建 3 个对象并输出学生总数。

作业 2：在 Student 类中增加 public static void showStudentCount()，输出学生总数，并在 Main 中通过 Student.showStudentCount() 调用。

作业 3：使用 HttpResult 创建 totalCount、successCount、slowCount 三个 private static 计数器。每创建对象，总数加 1；状态码为 2xx，成功数加 1；响应时间大于 500ms，慢请求数加 1。增加 public static void showStatistics() 输出统计结果。用 200/200、800/500、300/200、900/200、100/404 五组数据验证，预期总数 5、成功数 3、慢请求数 2。

> 思考题：为什么 showStatistics() 可以声明为 static，而 isSlow() 通常不应该声明为 static？

七、验收标准

能解释 static；能区分静态变量和普通成员变量、静态方法和普通方法；能通过类名访问静态成员；能完成请求计数；能解释为什么 int++ 在多线程场景下可能统计不准确。

提示：当前练习中的静态计数器是单线程教学示例。实际多线程性能统计需要使用线程安全的计数方式。